MSP Tool Stack Design & Standup — PSA, RMM & Patch Management

Implementation Solution Guide

Version 1.0.0 · 2026-06-27

Contents

Executive Summary — MSP Tool Stack Design & Standup — PSA, RMM & Patch Management 4

The business problem . . . 4

What this solution does . . . 4

Approach . . . 4

Business outcomes . . . 4

Investment . . . 5

1. Solution Overview — MSP Tool Stack Design & Standup — PSA, RMM & Patch Management 5

What this solution delivers . . . 5

Who this is for . . . 5

The standard MSP operating model . . . 5

Phase model . . . 6

Named objects you will create . . . 6

Scope and boundaries . . . 7

Outcomes . . . 7

2. Requirements — MSP Tool Stack Design & Standup — PSA, RMM & Patch Management 7

R1 — PSA platform selected, provisioned, and licensed . . . 7

R2 — RMM platform selected, provisioned, and licensed . . . 8

R3 — Agent deployment method confirmed . . . 8

R4 — Patch engine selected (RMM module or Intune / WUfB) . . . 8

R5 — Administrative accounts and a role model . . . 9

R6 — API keys / credentials for PSA↔RMM integration . . . 9

R7 — Naming and site taxonomy standard agreed . . . 9

R8 — Security and MFA on the tools . . . 9

R9 — A pilot client identified . . . 10

R10 — Network and agent connectivity . . . 10

R11 — Documentation / runbook home . . . 10

R12 — Secure secrets storage . . . 10

Requirements checklist . . . 11

3. Implementation — MSP Tool Stack Design & Standup — PSA, RMM & Patch Management 11

Stage A — Assess . . . 12

Stage B — Design . . . 12

Stage C — Stand up . . . 13

Stage D — Integrate . . . 14

Stage E — Onboard a client (pilot) . . . 15
5. Testing & Validation — MSP Tool Stack Design & Standup — PSA, RMM & Patch Management 15
V1 — PSA validation . . . 15
V2 — RMM validation . . . 16
V3 — Patch validation . . . 16
V4 — Integration validation . . . 17
V5 — Onboarding validation . . . 17
Automated validation . . . 17
Exit criteria . . . 18
6. Troubleshooting — MSP Tool Stack Design & Standup — PSA, RMM & Patch Management
PSA issues . . . 18
RMM issues . . . 18
Patch issues . . . 19
Integration issues . . . 19
Onboarding issues . . . 20
Escalation . . . 20
7. Best Practices & Operations — MSP Tool Stack Design & Standup — PSA, RMM & Patch Management 20
Operating cadence . . . 20
Standards that keep the stack healthy . . . 21
Expanding to the full client base . . . 22
Continuous improvement . . . 22
8. Deliverables Summary — MSP Tool Stack Design & Standup — PSA, RMM & Patch Management 22
Documentation . . . 22
Scripts . . . 23
Visual . . . 23
Named objects created by this solution . . . 23
Definition of done . . . 23
Appendix A — PowerShell Scripts 24

Executive Summary — MSP Tool Stack Design & Standup — PSA, RMM & Patch Management

The business problem

A managed-services practice lives or dies on the quality of the internal platforms it runs on. Three systems form that backbone: the **PSA** (the system of record for tickets, contracts, time, and billing), the **RMM** (the system of action that monitors and manages every endpoint), and a **Patch Management** engine (the system of compliance that keeps fleets current). When these are chosen ad hoc, configured inconsistently, and left unintegrated, the practice pays for it every day — alerts that never become tickets, time that never gets billed, assets nobody can account for, patches applied unevenly, and a new-client onboarding that is different (and error-prone) every time.

What this solution does

It designs and **stands up that backbone as one coherent program** — not a single tool wizard. It delivers:

- **Platform selection criteria** for the PSA, RMM, and patch engine, so the practice chooses on the right factors rather than on a sales demo.
- **A standardized operating model** — naming, site taxonomy, board/queue model, policy model, patch rings, and roles — captured as reusable, self-documenting standards.
- **The three integrations that make the stack behave as one system** — PSA↔RMM ticket sync, asset/CMDB feed, and alert→ticket flow.
- **A repeatable client-onboarding workflow** that stands up every new managed client the same way, validated against a pilot.
- **Validation with PASS/FAIL evidence** and a day-to-day operating cadence for sustained operation and expansion.

It is **vendor-neutral by design**: the architecture, standards, integration design, and onboarding workflow apply whichever PSA (e.g. ConnectWise PSA, Autotask, HaloPSA), RMM (e.g. NinjaOne, Datto RMM, ConnectWise Automate), and patch engine (the RMM module or Microsoft Intune / Windows Update for Business) the practice selects. Where a concrete console is named, exact navigation is provided; elsewhere the reader is pointed to the chosen vendor's console.

Approach

The program follows a disciplined seven-stage model — **Assess** → **Design** → **Stand up** → **Integrate** → **Onboard a client** → **Validate** → **Operate** — with named objects, a strict naming convention, and five PowerShell scripts that discover the current state, scaffold client onboarding, validate health, report coverage, and remediate drift.

Business outcomes

- **One source of truth** for clients, assets, tickets, and billing.

- **No dropped work or lost revenue** — alerts become tickets, time becomes invoices, automatically.
- **Consistent, fast onboarding** of every new managed client from a single template.
- **Governed, measurable patching** on a ring model instead of all-at-once risk.
- **A practice that scales** — the operating model and tooling are repeatable, documented, and auditable.

Investment

This is a flagship engagement (approximately $4,995) reflecting the multi-platform scope: it stands up and integrates the entire internal MSP operating backbone, not a single configuration task, and leaves the practice with a documented, validated, and repeatable operating model.

1. Solution Overview — MSP Tool Stack Design & Standup — PSA, RMM & Patch Management

What this solution delivers

A managed-services practice runs on a small set of internal platforms that, together, form its operating backbone. This solution designs and **stands up that backbone** end to end: the **PSA** (Professional Services Automation — ticketing, contracts, time tracking, billing), the **RMM** (Remote Monitoring & Management — agents, monitoring, scripting, automation), and a **Patch Management** engine (delivered either through the RMM or through Microsoft Intune / Windows Update for Business), plus the **integrations** that make them behave as one system (PSA↔RMM ticket sync, asset/CMDB feed, alert→ticket flow). It also delivers the **operating standards** (naming, site taxonomy, policies, roles) and a repeatable **client-onboarding workflow** so every new managed client is stood up the same way, every time.

This is a multi-platform **program**, not a single product wizard. The deliverables are vendor-neutral by design: the architecture, selection criteria, standards, integration design, onboarding workflow, and validation apply regardless of which PSA, RMM, or patch engine the practice selects. Where a concrete console is named (for example Microsoft Intune as the patch engine), exact navigation paths are given; where the platform is vendor-specific, you are directed to the chosen vendor's admin console and documentation for the product-specific clicks.

Who this is for

- A new or growing MSP that needs to assemble its core tool stack for the first time.
- An established practice consolidating fragmented or inherited tooling onto a clean, standardized model.
- An IT department adopting the managed-services operating model internally (multi-site or multi-business-unit).

The standard MSP operating model

The standard MSP operating model treats the stack as three cooperating planes:

1. **System of record (PSA).** Where work, contracts, time, and money live. The PSA is the single source of truth for tickets, clients (companies), configurations, agreements, and billing.
2. **System of action (RMM).** Where the practice sees and changes endpoints — agents on every managed device, monitoring, alerting, remote control, and scripted automation.
3. **System of compliance (Patch).** Where operating-system and third-party updates are governed on a ring-based schedule with measurable compliance, run by the RMM's patch module or by Intune / Windows Update for Business (WUfB).

The value of the stack is in the **seams**: alerts from the RMM become tickets in the PSA automatically; assets discovered by the RMM populate the PSA's CMDB; technician time logged against a ticket flows to billing. Designing those seams deliberately — rather than letting each tool operate in isolation — is what separates a professional practice from a collection of disconnected tools.

Phase model

This solution follows a seven-stage phase model. Each stage maps to the factory's Assess → Configure → Pilot → Validate → Expand lifecycle:

Stage	Purpose	Lifecycle
Assess	Inventory current tooling, contracts, and readiness; pick platforms against selection criteria.	Assess
Design	Define the operating standards: naming, site taxonomy, policy model, role model, integration design.	Configure
Stand up	Provision the PSA, RMM, and patch engine; apply the standards.	Configure
Integrate	Wire PSA↔RMM ticket sync, asset/CMDB feed, and alert→ticket flow.	Configure
Onboard a client	Run the standardized client-onboarding template against a pilot client.	Pilot
Validate	Verify each plane and each integration with PASS/FAIL evidence.	Validate
Operate	Adopt the day-to-day operating cadence; expand to the full client base.	Expand

Named objects you will create

This solution uses a strict – – naming convention, reused verbatim across every object so the stack is self-documenting. The core objects created are:

- **PSA boards/queues:** `Service Desk – Reactive – Triage`, `Service Desk – Reactive – Dispatch`, `Projects – Onboarding – Delivery`, `NOC – Alerts – Automated`.
- **RMM sites/policies:** `Client Site – {ClientName} – Production`, `Monitoring Policy – Workstation – Standard`, `Monitoring Policy – Server – Standard`, `Automation Policy – Maintenance – Nightly`.
- **Patch rings:** `Patch Ring – Pilot – Stage0`, `Patch Ring – Early – Stage1`, `Patch Ring – Broad – Stage2`, `Patch Ring – Critical Servers – Stage3`.
- **Integration:** `Integration – PSA-RMM – Bi-Directional`.
- **Client-onboarding template:** `Onboarding Template – Managed Client – Standard`.

Every object created later in this solution reuses these exact names.

Scope and boundaries

In scope: platform selection criteria; provisioning and baseline configuration of the PSA, RMM, and patch engine; the operating standards; the three core integrations; a repeatable client-onboarding workflow; validation and an operating cadence.

Out of scope: selecting or negotiating client-facing service offerings and pricing; security tooling beyond securing the stack itself (EDR, SIEM, email security are separate solutions); detailed accounting/general-ledger configuration in the PSA's billing module beyond enabling the time→invoice flow; backup/BCDR tooling.

Outcomes

On completion the practice has a standardized, integrated tool stack: one PSA system of record, one RMM system of action, one governed patch engine, three live integrations, a documented operating standard, and a proven onboarding workflow that stands up a new managed client repeatably and verifiably.

2. Requirements — MSP Tool Stack Design & Standup — PSA, RMM & Patch Management

Do not assume any prerequisite is already in place. Work through each requirement below. For each item, confirm the **Description**, understand **Why it is required**, run the **How to verify** check, and if the check fails, follow **How to implement if missing** before proceeding to implementation. Requirements R1–R12 are specific to this solution; the category-wide requirements common to every Managed solution are in `global-requirements-managed.md` and must also be satisfied.

R1 — PSA platform selected, provisioned, and licensed

- **Description:** A Professional Services Automation platform (for example ConnectWise PSA, Datto Autotask, or HaloPSA) provisioned with a tenant/instance and enough licensed seats for the technical

and administrative staff who will use it.

• **Why required:** The PSA is the system of record for tickets, companies, agreements, time, and billing. Without it there is no single source of truth and no place for RMM alerts to land as tickets.

• **How to verify:** Sign in to the PSA admin console as an administrator and confirm the instance is active and the licensed seat count covers all named users (console → System/Admin → Licensing or Members).

• **How to implement if missing:** Run the platform selection criteria in `03-implementation.md` (Stage Assess), procure the platform, and request the tenant/instance from the vendor. Record the chosen platform in the documentation home (R12).

R2 — RMM platform selected, provisioned, and licensed

• **Description:** A Remote Monitoring & Management platform (for example NinjaOne, Datto RMM, or ConnectWise Automate) provisioned with a tenant and licensed endpoint capacity.

• **Why required:** The RMM is the system of action — it deploys agents, monitors endpoints, alerts, and runs scripted automation across every managed device.

• **How to verify:** Sign in to the RMM admin console and confirm the tenant is active and the licensed endpoint count covers the expected device population (console → Administration → Licensing).

• **How to implement if missing:** Select against the criteria in `03-implementation.md`, procure, and provision the tenant. Confirm the licensing model (per-endpoint vs per-technician) matches the practice's growth plan.

R3 — Agent deployment method confirmed

• **Description:** A confirmed, repeatable method to deploy the RMM agent to managed endpoints (group policy / Intune app deployment / deployment script / network-installer URL).

• **Why required:** Endpoints are invisible to the stack until the agent is installed; the onboarding workflow depends on a reliable bulk-deployment path.

• **How to verify:** Confirm the RMM provides an agent installer and a deployment package, and that at least one deployment channel (Intune, GPO, or RMM network discovery) is available in the target environment.

• **How to implement if missing:** Generate the agent installer from the RMM (console → Administration → Agents/Downloads), and prepare a deployment package for the chosen channel (for Intune: package as a Win32 app; for GPO: an MSI assigned via a software-installation policy).

R4 — Patch engine selected (RMM module or Intune / WUfB)

• **Description:** A decision on which engine governs operating-system and third-party patching: the RMM's native patch module, or Microsoft Intune with Windows Update for Business (WUfB).

• **Why required:** Patching is the system of compliance; the engine choice determines where rings, schedules, and compliance reporting live and must be settled before ring design.

• **How to verify:** Confirm the selected engine is available — for the RMM module, that the license includes patch management; for Intune/WUfB, that you have a Microsoft Entra tenant with Intune

licensing and devices that are Entra-joined or hybrid-joined and enrolled in Intune.

• **How to implement if missing:** If using the RMM module, enable it in the RMM license. If using Intune/WUfB, ensure Intune licensing (Microsoft Intune Plan 1 or a bundle that includes it) and device enrollment. See `03-implementation.md` Stage Design for the selection criteria.

R5 — Administrative accounts and a role model

• **Description:** Dedicated administrator accounts for the PSA, RMM, and patch engine, plus a defined set of roles (administrator, technician, dispatcher, read-only/reporting).

• **Why required:** Least-privilege role separation prevents accidental or malicious damage and is required for the onboarding workflow to assign technicians correctly.

• **How to verify:** In each console confirm at least one named administrator account exists (not a shared generic login) and that role definitions exist (PSA → Security Roles; RMM → Roles/Permissions).

• **How to implement if missing:** Create named admin accounts; define the four roles above; assign existing users to the least-privilege role that lets them do their job.

R6 — API keys / credentials for PSA↔RMM integration

• **Description:** API keys, OAuth client credentials, or integration service accounts in both the PSA and the RMM, scoped to the integration's needs.

• **Why required:** The PSA↔RMM ticket sync, asset/CMDB feed, and alert→ticket flow all authenticate via these credentials; without them the seams cannot be built.

• **How to verify:** Confirm an API/integration section exists in each console and that you can generate a key/credential there (PSA → System → API/Integrations; RMM → Administration → API).

• **How to implement if missing:** Generate a dedicated integration credential in each tool, named for the integration (`Integration – PSA-RMM – Bi-Directional`), scoped to least privilege, and store both in the secrets store (see global requirements). Never embed keys in scripts.

R7 — Naming and site taxonomy standard agreed

• **Description:** An agreed `–  –` naming convention and a site taxonomy mapping each managed client to one RMM site and one PSA company.

• **Why required:** A consistent taxonomy makes the stack self-documenting, prevents duplicate/ambiguous objects, and is the backbone of the onboarding template.

• **How to verify:** Confirm the convention from `01-solution-overview.md` is documented and approved, and that no conflicting legacy naming exists in either tool.

• **How to implement if missing:** Adopt the convention as written; if legacy objects exist, plan a rename/migration during Stage Design.

R8 — Security and MFA on the tools

• **Description:** Multi-factor authentication enforced on the PSA, RMM, and patch engine for all human accounts, and SSO configured where supported.

• **Why required:** These tools have privileged, cross-client reach; a compromised admin login can affect every managed client. MFA is non-negotiable.

• **How to verify:** In each console confirm MFA is enforced for all users (PSA → Security → MFA/SSO; RMM → Administration → Security/2FA; Intune via Entra Conditional Access).

• **How to implement if missing:** Enforce MFA for all users in each tool. Where the tool supports SSO via Entra ID, configure it and apply a Conditional Access policy requiring MFA.

R9 — A pilot client identified

• **Description:** One real (or representative test) managed client designated as the pilot for the onboarding workflow and validation.

• **Why required:** The onboarding workflow and integrations must be validated against a real client before the model is expanded to the full base.

• **How to verify:** Confirm a pilot client is named, has consented to (or is a safe test stand-in for) being onboarded, and has at least a few endpoints available.

• **How to implement if missing:** Select a low-risk, cooperative client or stand up an internal test company/site with a handful of test devices.

R10 — Network and agent connectivity

• **Description:** Outbound connectivity from managed endpoints to the RMM and patch sources (HTTPS to vendor cloud endpoints; Windows Update / WSUS / Delivery Optimization for patches).

• **Why required:** Agents must reach the RMM cloud to report and be managed; patch engines must reach update sources to download and report compliance.

• **How to verify:** From a representative endpoint, confirm it can reach the RMM's documented hostnames/ports and Microsoft Update endpoints (and any required allow-list URLs).

• **How to implement if missing:** Allow-list the RMM vendor's documented URLs/IPs on perimeter firewalls and proxies; ensure Windows Update endpoints are reachable. Use the discovery script to flag blocked endpoints.

R11 — Documentation / runbook home

• **Description:** A documentation location (IT documentation platform or wiki) where the operating standards, integration details, and onboarding runbook live.

• **Why required:** The stack is only repeatable if its standards and runbooks are written down where the whole team can find them.

• **How to verify:** Confirm a documentation home exists and the team has access.

• **How to implement if missing:** Stand up or designate a documentation home and create a section for the MSP operating standard.

R12 — Secure secrets storage

• **Description:** A secure secrets store (password manager / secrets vault) for the integration API keys, admin credentials, and service accounts.

• **Why required:** The integration credentials are highly privileged; storing them in scripts, spreadsheets, or chat is an immediate, fleet-wide risk.

• **How to verify:** Confirm a secrets store is in use and that the integration credentials are stored there, not in code.

• **How to implement if missing:** Adopt a secrets store; migrate all credentials into it; rotate any credential that was previously stored insecurely.

Requirements checklist

#	Requirement	Verified
R1	PSA platform selected, provisioned, licensed	■
R2	RMM platform selected, provisioned, licensed	■
R3	Agent deployment method confirmed	■
R4	Patch engine selected (RMM or Intune/WUfB)	■
R5	Admin accounts + role model	■
R6	API keys for PSA↔RMM integration	■
R7	Naming / site taxonomy standard	■
R8	Security + MFA on the tools	■
R9	Pilot client identified	■
R10	Network / agent connectivity	■
R11	Documentation / runbook home	■
R12	Secure secrets storage	■

3. Implementation — MSP Tool Stack Design & Standup — PSA, RMM & Patch Management

This is the build guide for the full program. Work the phases in order: **Assess** → **Design** → **Stand up** → **Integrate** → **Onboard a client** → **Validate** → **Operate** (Validate and Operate are detailed in `05-testing.md` and `07-best-practices.md`; the core build is Assess through Onboard here). Where a concrete console is named, full navigation paths are given. Where the platform is vendor-specific, the step names the **vendor admin console** → **section** → **tab** to open and directs you to that vendor's documentation for product-specific clicks. Every object you create reuses the `–` `–` names defined in `01-solution-overview.md` verbatim.

Do not start until the requirements in `02-requirements.md` and `global-requirements-managed.md` are verified.

Stage A — Assess

Step 1 — Inventory the current tooling and integration readiness

Run `MSPSTACK-Discovery.ps1` (see `scripts/`) to capture what exists today: any PSA/RMM/patch tooling, installed agents, reachable update sources, and integration-credential readiness. Save the output as the baseline. If the practice has no tooling yet, the script confirms a greenfield start.

Step 2 — Select the PSA platform against criteria

Score candidate PSA platforms (for example ConnectWise PSA, Datto Autotask, HaloPSA) against:

- **Ticketing depth:** boards/queues, SLAs, dispatch, time entry against tickets.
- **Contracts & billing:** agreement types (managed, block-hours, per-device), time→invoice flow.
- **Integration breadth:** native or documented API integration with the candidate RMM and patch engine.
- **CMDB / configurations:** ability to hold per-client configuration items fed from the RMM.
- **Total cost & growth model:** per-seat vs per-endpoint pricing aligned to the growth plan.

Record the decision and rationale in the documentation home. Confirm R1.

Step 3 — Select the RMM platform against criteria

Score candidate RMM platforms (for example NinjaOne, Datto RMM, ConnectWise Automate) against:

- **Agent footprint & OS coverage:** Windows/macOS/Linux/server support, agent reliability.
- **Monitoring & alerting:** built-in monitors, alert thresholds, escalation, alert→ticket capability.
- **Scripting & automation:** scripting engine, scheduled automation, software deployment.
- **Patch capability:** native OS + third-party patch module (if RMM is the chosen patch engine).
- **PSA integration:** native bi-directional integration with the chosen PSA.

Record the decision. Confirm R2.

Step 4 — Select the patch engine

Decide between the **RMM patch module** and **Intune / Windows Update for Business (WUfB)**:

- Choose the **RMM module** when you want a single pane for monitoring and patching, mixed-OS coverage, and third-party app patching driven by the RMM.
- Choose **Intune / WUfB** when devices are Entra-joined/enrolled and you want Microsoft-native update rings, Conditional Access integration, and Update Compliance reporting.

Record the decision. Confirm R4.

Stage B — Design

Step 5 — Lock the naming and site taxonomy

Adopt the `– –` convention from `01-solution-overview.md`. Map every managed client to **one RMM site** (`Client Site – {ClientName} – Production`) and **one PSA company** of the same client name. Document the mapping in the documentation home. Confirm R7.

Step 6 — Design the PSA board/queue model

Plan these boards/queues (created in Stage C):

- `Service Desk – Reactive – Triage` — entry point for all inbound and alert-generated tickets.
- `Service Desk – Reactive – Dispatch` — triaged tickets assigned to technicians.
- `Projects – Onboarding – Delivery` — onboarding and project work.
- `NOC – Alerts – Automated` — RMM alert tickets that auto-resolve when the alert clears.

Step 7 — Design the RMM policy model

Plan these policies (created in Stage C):

- `Monitoring Policy – Workstation – Standard` — baseline workstation monitors (disk, CPU, services, AV health).
- `Monitoring Policy – Server – Standard` — server monitors (services, event logs, disk, backup status).
- `Automation Policy – Maintenance – Nightly` — scheduled nightly maintenance (temp cleanup, health checks, reboots where permitted).

Step 8 — Design the patch rings

Plan a ring-based deployment model (created in Stage C against the chosen engine):

- `Patch Ring – Pilot – Stage0` — small canary set; updates land first, 0-day deferral.
- `Patch Ring – Early – Stage1` — early adopters; ~3-day deferral.
- `Patch Ring – Broad – Stage2` — the bulk of workstations; ~7-day deferral.
- `Patch Ring – Critical Servers – Stage3` — servers; longest deferral, scheduled maintenance windows only.

Step 9 — Design the role model

Define the roles from R5 in each tool: administrator, technician, dispatcher, read-only/reporting. Map staff to least-privilege roles.

Stage C — Stand up

Step 10 — Provision and baseline the PSA

In the **PSA admin console**, sign in as administrator and create the boards/queues from Step 6. (In ConnectWise PSA: *System → Setup Tables → Service Board* to create each board; in Autotask: *Admin → Features & Settings → Service Desk (Tickets) → Queues*; in HaloPSA: *Configuration → Tickets → Ticket Types / Teams*.) Refer to the chosen vendor's documentation for the exact field-by-field setup. Then:

- Create the SLA(s) and attach them to `Service Desk – Reactive – Triage`.
- Create agreement types for managed, block-hours, and per-device contracts.
- Enable time entry against tickets so technician time can flow to billing.

Step 11 — Provision and baseline the RMM

In the **RMM admin console**, sign in as administrator and:

- Create the monitoring and automation policies from Step 7 (NinjaOne: *Administration → Policies*; Datto RMM: *Account → Policies*; ConnectWise Automate: *Automation → Templates/Scripts and Monitors*). Refer to the vendor docs for monitor-by-monitor setup.
- Confirm the agent installer and the deployment package from R3 are ready.
- Create the role definitions from Step 9 (RMM → Administration → Roles/Permissions) and enforce MFA (R8).

Step 12 — Stand up the patch engine

If the RMM is the patch engine: in the RMM patch section, create the four patch rings from Step 8 as patch policies, each with its deferral/approval rules and maintenance window. Bind rings to device groups (created per client during onboarding).

If Intune / WUfB is the patch engine: in the **Microsoft Intune admin center** (`https://intune.microsoft.com`):

1. Create quality-update rings at **Devices** → **Manage updates** → **Windows updates** → **Update rings** → **Create profile**. Create one profile per ring: `Patch Ring – Pilot – Stage0`, `Patch Ring – Early – Stage1`, `Patch Ring – Broad – Stage2`, `Patch Ring – Critical Servers – Stage3`. Set the **Quality update deferral period (days)** to 0 / 3 / 7 / 7 respectively and configure deadline/grace settings, then assign each profile to its corresponding device group.
2. Create the assignment groups at **Microsoft Entra admin center** → **Groups** → **New group** (or in Intune at **Groups** → **New group**) named to match each ring's scope, then return to each update ring profile's **Assignments** tab and assign the matching group.
3. (Optional) Configure **feature-update** profiles at **Devices** → **Manage updates** → **Windows updates** → **Feature updates** → **Create profile** to pin the target Windows feature version per ring.

Stage D — Integrate

Step 13 — Create the PSA↔RMM integration

Using the API credentials from R6 (retrieved from the secrets store), configure the bi-directional integration named `Integration – PSA-RMM – Bi-Directional`:

1. In the **RMM admin console** → **Administration** → **Integrations** (or the PSA's integration marketplace), select the native PSA integration and authenticate with the integration credential. Refer to the vendor's integration setup guide for the exact field mapping screens.
2. Configure **alert→ticket flow:** map RMM alerts to create tickets on `NOC – Alerts – Automated`, and configure auto-close so a ticket resolves when its alert clears.

3. Configure **asset/CMDB feed:** map RMM devices to PSA configuration items so the RMM populates the PSA CMDB (R-aligned with GM3).

4. Configure **ticket sync:** ensure ticket status/notes sync between the tools where the vendor supports it.

Step 14 — Map sites to companies

In the integration's mapping screen, link each `Client Site – {ClientName} – Production` RMM site to its matching PSA company so alerts, assets, and tickets attribute to the correct client.

Stage E — Onboard a client (pilot)

Step 15 — Run the onboarding template against the pilot client

Use `MSPSTACK-Set-ClientOnboarding.ps1` (run with `-WhatIf` first) to scaffold the pilot client (R9) per `Onboarding Template – Managed Client – Standard`:

1. Create the PSA company and the RMM site `Client Site – {ClientName} – Production`.
2. Create the client's device groups and bind the monitoring/automation policies and patch rings.
3. Deploy the RMM agent via the method from R3 to the client's endpoints.
4. Confirm assets flow into the PSA CMDB and that a test alert creates a ticket on `NOC – Alerts – Automated`.
5. Record the onboarding in the documentation home and the PSA configurations module.

Re-run with `-Apply` (script default is read-only/`-WhatIf`) once the dry-run output is correct.

After onboarding the pilot, proceed to `05-testing.md` for Stage Validate, then `07-best-practices.md` for Stage Operate.

5. Testing & Validation — MSP Tool Stack Design & Standup — PSA, RMM & Patch Management

This is **Stage Validate**. Verify each plane (PSA, RMM, patch) and each integration with explicit PASS/FAIL evidence before expanding the model to the full client base. Run `MSPSTACK-Validation.ps1` for the automated checks and complete the manual checks below. Capture evidence (screenshots, exported CSVs from `MSPSTACK-Report.ps1`) in the documentation home.

A stage **PASSES** only when every check in its table passes. Any FAIL must be remediated (see `06-troubleshooting.md` and `MSPSTACK-Remediation.ps1`) and re-tested before proceeding.

V1 — PSA validation

Check	Method	PASS criteria
Boards/queues exist	PSA console → service boards/queues list	`Service Desk – Reactive – Triage`, …– `Dispatch`, `Projects – Onboarding – Delivery`, `NOC – Alerts – Automated` all present
SLA attached	Open `Service Desk – Reactive – Triage`	SLA is attached and active
Time→billing flow	Create a test ticket, log time, generate a draft invoice	Logged time appears on a billable agreement
Roles defined	PSA → Security/Roles	Administrator, technician, dispatcher, read-only roles exist
MFA enforced	PSA → Security → MFA/SSO	MFA required for all users

V2 — RMM validation

Check	Method	PASS criteria
Policies exist	RMM → Policies	`Monitoring Policy – Workstation – Standard`, …– `Server – Standard`, `Automation Policy – Maintenance – Nightly` present
Agent reporting	RMM → Devices	Pilot endpoints show online with recent check-in
Monitors firing	Trigger a test condition (e.g. stop a monitored service)	An alert is raised on the endpoint
Automation runs	Run `Automation Policy – Maintenance – Nightly` on demand	Completes with success result
MFA enforced	RMM → Administration → Security	MFA required for all users

V3 — Patch validation

Check	Method	PASS criteria
Rings exist	RMM patch section, or Intune → Devices → Manage updates → Windows updates → Update rings	`Patch Ring – Pilot – Stage0` … `– Critical Servers – Stage3` all present
Deferrals correct	Open each ring	Deferral = 0 / 3 / 7 / 7 days respectively

Check	Method	PASS criteria
Assignments correct	Each ring's Assignments tab	Each ring assigned to its matching device group
Compliance reporting	RMM patch report, or Intune → Devices → Monitor → Windows Update reports	Pilot devices report patch status

V4 — Integration validation

Check	Method	PASS criteria
Integration named & active	RMM/PSA integrations list	`Integration – PSA-RMM – Bi-Directional` shows connected
Alert→ticket	Trigger a test RMM alert	A ticket appears on `NOC – Alerts – Automated` for the correct company
Auto-close	Clear the test alert	The ticket auto-resolves
Asset/CMDB feed	RMM → discover an asset	The asset appears as a PSA configuration item under the right company
Site↔company mapping	Onboarded pilot site	`Client Site – {ClientName} – Production` maps to the matching PSA company

V5 — Onboarding validation

Check	Method	PASS criteria
Site + company created	RMM/PSA	Pilot client has one site and one company, correctly named
Policies/rings bound	RMM device groups	Pilot groups bound to monitoring/automation policies and patch rings
Agents deployed	RMM → Devices	Pilot endpoints report in
End-to-end flow	Trigger alert → ticket → resolve	Full alert→ticket→close cycle works for the pilot client

Automated validation

Run `MSPSTACK-Validation.ps1` to evaluate the machine-checkable items (object presence, agent check-in, integration connectivity, ring deferrals) and emit a PASS/FAIL summary. Run

`MSPSTACK-Report.ps1` to export stack-coverage and onboarding-status CSVs as validation evidence. Treat the automated output as supporting evidence alongside the manual checks — both must pass.

Exit criteria

Stage Validate is complete when V1–V5 all PASS and the evidence is filed in the documentation home. Only then proceed to Stage Operate (`07-best-practices.md`) and expand onboarding to the full client base.

6. Troubleshooting — MSP Tool Stack Design & Standup — PSA, RMM & Patch Management

Common failures encountered while standing up and operating the stack, with causes and fixes. Many fixes are automated in `MSPSTACK-Remediation.ps1` (run `-WhatIf` first, then `-Apply`). Re-run the relevant check in `05-testing.md` after any fix.

PSA issues

Boards/queues missing or misnamed

- **Symptom:** Validation V1 fails — a board/queue is absent or named inconsistently.
- **Cause:** Created out of convention, or not created at all.
- **Fix:** Recreate using the exact `– –` names (e.g. `Service Desk – Reactive – Triage`). In the PSA console's service board/queue section, rename or create as needed.

Time not flowing to billing

- **Symptom:** Logged technician time does not appear on a draft invoice.
- **Cause:** Time entry against tickets is disabled, or the ticket is not tied to a billable agreement.
- **Fix:** Enable time entry on tickets; attach the ticket's company to a billable agreement type; confirm the time entry is marked billable.

RMM issues

Agent not reporting / endpoint offline

- **Symptom:** A device shows offline or never checks in after deployment.
- **Cause:** Agent not installed, blocked outbound connectivity, or wrong site assignment.
- **Fix:** Confirm the agent is installed; verify the endpoint can reach the RMM's documented hostnames/ports (R10); confirm the device is in `Client Site – {ClientName} – Production`. Use `MSPSTACK-Discovery.ps1` to flag connectivity gaps.

Monitors not firing / alert noise

- **Symptom:** No alert on a real condition, or too many low-value alerts.

• **Cause:** Policy not bound to the device group, or thresholds mis-tuned.

• **Fix:** Confirm `Monitoring Policy – Workstation – Standard` / …– `Server – Standard` are bound to the right groups; tune thresholds. Re-test by triggering a known condition.

Nightly automation failing

• **Symptom:** `Automation Policy – Maintenance – Nightly` reports failures.

• **Cause:** Insufficient permissions, locked files, or reboots blocked by policy.

• **Fix:** Review the automation log on a failing endpoint; adjust the automation's run context/permissions; honor reboot-suppression where required.

Patch issues

Devices not receiving updates

• **Symptom:** Patch ring shows devices stuck or non-compliant.

• **Cause:** Device not in the ring's assigned group, blocked update source, or deferral not yet elapsed.

• **Fix:** Confirm the device is in the ring's assignment group; verify reachability of Windows Update / WSUS / Delivery Optimization (R10); confirm the deferral window. For Intune: *Devices → Manage updates → Windows updates → Update rings → (open ring) → Assignments* and *Devices → Monitor → Windows Update reports*.

Wrong deferral on a ring

• **Symptom:** Validation V3 fails — a ring's deferral does not match the design (0/3/7/7).

• **Cause:** Misconfiguration at ring creation.

• **Fix:** Open the ring and correct the **Quality update deferral period (days)**. In Intune: *Devices → Manage updates → Windows updates → Update rings → (open ring) → Properties → Settings*.

Integration issues

Integration shows disconnected

• **Symptom:** `Integration – PSA-RMM – Bi-Directional` is not connected.

• **Cause:** Expired/rotated API credential, or insufficient credential scope.

• **Fix:** Retrieve the current credential from the secrets store (R12); re-authenticate the integration; confirm the credential's scope covers tickets, assets, and alerts. Rotate if compromised.

Alerts not creating tickets

• **Symptom:** RMM alerts fire but no ticket appears on `NOC – Alerts – Automated`.

• **Cause:** Alert→ticket mapping not configured, or site↔company mapping missing.

• **Fix:** In the integration's mapping screen, confirm alerts map to `NOC – Alerts – Automated` and that `Client Site – {ClientName} – Production` maps to the matching PSA company. Trigger a test alert to confirm.

Tickets not auto-closing

- **Symptom:** Alert clears but the ticket stays open.
- **Cause:** Auto-close on alert-clear not enabled.
- **Fix:** Enable auto-close in the alert→ticket mapping so a resolved alert closes its ticket.

Assets not appearing in the CMDB

- **Symptom:** RMM-discovered devices are not configuration items in the PSA.
- **Cause:** Asset/CMDB feed not enabled or device-to-CI mapping missing.
- **Fix:** Enable the asset feed in the integration; confirm device-to-configuration-item mapping; let the next sync run, then re-check the PSA company's configurations.

Onboarding issues

Onboarding script dry-run shows unexpected objects

- **Symptom:** `MSPSTACK-Set-ClientOnboarding.ps1 -WhatIf` would create duplicates or wrong names.
- **Cause:** Legacy objects exist, or the client name input is inconsistent.
- **Fix:** Reconcile against the naming standard; correct the client-name parameter; only run `-Apply` once the dry-run output is correct.

Escalation

If a fix is not covered here: re-run `MSPSTACK-Discovery.ps1` and `MSPSTACK-Validation.ps1` to localize the failing plane, consult the chosen vendor's admin console and documentation for product-specific behavior, and route any fleet-wide change through change control (GM4) before applying.

7. Best Practices & Operations — MSP Tool Stack Design & Standup — PSA, RMM & Patch Management

This is **Stage Operate** — the day-to-day operating cadence once the stack is stood up, integrated, and validated, plus the standards that keep it healthy as you **expand** to the full client base.

Operating cadence

Daily

- Work `Service Desk – Reactive – Triage` to zero: every inbound and alert-generated ticket triaged and dispatched to `Service Desk – Reactive – Dispatch`.
- Review `NOC – Alerts – Automated` for tickets that did not auto-close; investigate noisy or stuck alerts.
- Confirm `Automation Policy – Maintenance – Nightly` completed across all sites; investigate failures.

Weekly

- Review patch compliance per ring; chase devices missing from `Patch Ring – Broad – Stage2`.
- Reconcile the CMDB: confirm RMM-discovered assets match PSA configuration items per client.
- Review alert→ticket noise and tune thresholds on `Monitoring Policy – Workstation – Standard` / …`– Server – Standard`.

Monthly

- Verify time→billing: all billable technician time is captured against the right agreement.
- Audit MFA and role assignments across all three tools (R5, R8).
- Rotate or re-verify the `Integration – PSA-RMM – Bi-Directional` credentials per the secrets-store policy.

Standards that keep the stack healthy

Naming discipline

Every new object — board, queue, site, policy, ring, group — uses the `–` `–` convention verbatim. Never create ad-hoc one-off objects; if a new pattern is needed, add it to the standard in the documentation home first.

One site, one company, per client

Each managed client maps to exactly one `Client Site – {ClientName} – Production` RMM site and one PSA company. Never split a client across multiple sites or merge clients into one site — it breaks attribution of alerts, assets, and billing.

Ring-based patching, never all-at-once

Patches always flow Pilot → Early → Broad → Critical Servers. Never push an update straight to all devices. Watch `Patch Ring – Pilot – Stage0` for regressions before the wave reaches `Patch Ring – Broad – Stage2`.

Integration as the source of truth for assets

Let the RMM be the discovery source and the PSA be the system of record; the `Integration – PSA-RMM – Bi-Directional` feed keeps them aligned. Do not hand-maintain assets in the PSA — fix discovery at the RMM instead.

Change control on fleet-wide changes

Any change to a shared policy, ring deferral, or the integration affects every client at once. Route these through change control (GM4): pilot on one client/site, validate, then expand.

Least privilege and MFA always

Keep the four roles tight; grant the minimum each user needs. MFA stays enforced on all three tools for every human account, with SSO via Entra where supported.

Expanding to the full client base

Onboard each remaining client by re-running `Onboarding Template – Managed Client – Standard` (via `MSPSTACK-Set-ClientOnboarding.ps1`, `-WhatIf` then `-Apply`). Onboard in waves, validating each wave with `MSPSTACK-Validation.ps1` before the next. Keep `MSPSTACK-Report.ps1` output as the running record of stack coverage and onboarding status.

Continuous improvement

- Track recurring alert types and convert them into RMM automation that self-remediates before a ticket is needed.
- Review SLA performance on `Service Desk – Reactive – Triage` and adjust dispatch/staffing.
- Re-run `MSPSTACK-Discovery.ps1` periodically to catch unmanaged endpoints and integration drift.
- Keep the operating standard, integration details, and onboarding runbook current in the documentation home — the stack is only as repeatable as its documentation.

8. Deliverables Summary — MSP Tool Stack Design & Standup — PSA, RMM & Patch Management

This solution delivers a complete program to design and stand up the core internal MSP platform stack — PSA, RMM, and Patch Management — with their integrations, operating standards, and a repeatable client-onboarding workflow. Below is everything included.

Documentation

Document	Purpose
`01-solution-overview.md`	The standard MSP operating model, phase model, named objects, scope.
`02-requirements.md`	Solution requirements R1–R12 (Description / Why / Verify / Implement).
`global-requirements-managed.md`	Category-wide requirements for every Managed solution (GM1–GM7).
`03-implementation.md`	The build guide: Assess → Design → Stand up → Integrate → Onboard a client.
`05-testing.md`	Stage Validate — PASS/FAIL checks for each plane and integration.
`06-troubleshooting.md`	Common failures and fixes across PSA, RMM, patch, and integration.
`07-best-practices.md`	Stage Operate — operating cadence, standards, and expansion.
`executive-summary.md`	Business-level summary for decision-makers.
`08-deliverables-summary.md`	This document.

Scripts

Five runnable PowerShell scripts, generic/REST-API-oriented where vendor APIs apply and Microsoft Graph where Intune patch applies. Each carries a `.SYNOPSIS` with PURPOSE / HOW TO RUN / EXPECTED RESULTS and neutral `.NOTES`.

Script	Purpose
`MSPSTACK-Discovery.ps1`	Inventory current tooling and integration readiness; baseline a greenfield or existing environment.
`MSPSTACK-Validation.ps1`	Verify PSA / RMM / patch and integration health; emit PASS/FAIL.
`MSPSTACK-Report.ps1`	Export stack-coverage and onboarding-status to CSV (validation/operating evidence).
`MSPSTACK-Remediation.ps1`	Apply safe fixes for common drift; read-only / `-WhatIf` by default, `-Apply` to act.
`MSPSTACK-Set-ClientOnboarding.ps1`	Scaffold a new managed client's site, policies, groups, and rings per the onboarding template; `-WhatIf` by default.

Visual

Asset	Purpose
`MSPSTACK-hero.png`	Hero image for the solution listing.

Named objects created by this solution

All objects use the `–  –` convention:

- **PSA boards/queues:** `Service Desk – Reactive – Triage`, `Service Desk – Reactive – Dispatch`, `Projects – Onboarding – Delivery`, `NOC – Alerts – Automated`.
- **RMM sites/policies:** `Client Site – {ClientName} – Production`, `Monitoring Policy – Workstation – Standard`, `Monitoring Policy – Server – Standard`, `Automation Policy – Maintenance – Nightly`.
- **Patch rings:** `Patch Ring – Pilot – Stage0`, `Patch Ring – Early – Stage1`, `Patch Ring – Broad – Stage2`, `Patch Ring – Critical Servers – Stage3`.
- **Integration:** `Integration – PSA-RMM – Bi-Directional`.
- **Onboarding template:** `Onboarding Template – Managed Client – Standard`.

Definition of done

The stack is delivered when: the PSA, RMM, and patch engine are stood up with the standard objects; the three integrations are live; the pilot client is onboarded via the template; Stage Validate (V1–V5) passes with filed evidence; and the operating standard and runbook are documented in the documentation home.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

MSPSTACK-Discovery.ps1

MSPSTACK-Remediation.ps1

MSPSTACK-Report.ps1

MSPSTACK-Set-ClientOnboarding.ps1

MSPSTACK-Validation.ps1